STUDENT DROPOUT AND SUCCESS PREDICTION

A CORNERSTONE PROJECT REPORT

Submitted in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

Submitted by

R V S V Karthik – 24B11AI373

K Narayana Swamy – 24B11AI196

M Dileep – 24B11AI233

S Eliyas Ahamad – 24B11AI393

Under the Guidance of

M. Pravallika

ABSTRACT

The rapid growth of student data in higher education institutions has created significant opportunities to apply data mining techniques for dropout prediction and academic decision support. This project analyzes two student datasets – a performance dataset (4,424 × 13) and an activity dataset (4,424 × 22) – collected from a higher education institution. Both datasets contained substantial missing values (300–480 per column) which were handled using mean imputation for numerical attributes and mode imputation for categorical attributes. After cleaning and merging using a common identifier, 3,967 valid student records with 35 features were produced. The merged dataset was analyzed using Python for exploratory analysis and WEKA for data mining. Classification algorithms including Decision Tree (J48) and Random Forest were applied. Random Forest achieved the highest accuracy of 73.6% (cross-validation: 75.07%), outperforming Decision Tree which achieved 65.5%. Although overall accuracy was reasonable, class imbalance caused poor recall for the Enrolled class, highlighting the need for imbalance handling in future work. Clustering and association rule mining revealed structural groupings and demographic co-occurrence patterns within the student population.

Keywords: Decision Trees, Dropout Prediction, Predictive Analysis, Classification Algorithms, Clustering Techniques, Association Rule Mining, Data Visualization, WEKA.

Index

Chapter .No	Table of Content	Page No.
1	**Introduction**	1
	1.1 Introduction	1
	1.2 Problem Statement	3
	1.3 Objectives of the Project	4
	1.4 Scope of the Project	5
2	**System Overview**	6
	2.1 Existing System	7
	2.2 Limitations of Existing System	8
	2.3 Proposed System	9
	2.4 Software Requirements	10
	2.5 Hardware Requirements	11
3	**System Implementation**	13
	3.1 Overview of Development Tools	14
	3.1.1 Jupyter Notebook9	14
	3.1.2 Weka Installation	22
4	**System Architecture**	25
5	**Data Preprocessing**	28
	5.1 Introduction	29
	5.2 Handling Missing Values	22

	5.3 Data Cleaning	30
	5.4 Noise Removal and Range Validation	30
	5.5 Data Transformation	31
	5.6 Data Visualization	31
	5.7 Data Normalization	32
	5.8 Data Export	32
	5.9 Summary	32
6	**Methodology and Algorithms**	33
	6.1 Introduction	34
	6.2 Classification Techniques	34
	6.2.1 Decision Tree (J48)	34
	6.2.2 Random Forest	35
	6.3 Clustering Techniques	35
	6.3.1 K-Means Clustering	35
	6.3.2 Hierarchical Clustering	36
	6.4 Association Rule Mining	36
	6.4.1 Apriori Algorithm	36
	6.4.1 Apriori Algorithm	37
	6.5 Model Evaluation	37
7	**Results and Analysis**	38
	7.1 Introduction	39
	7.2 Experimental Setup	39

	7.3 Classification Analysis	39
	7.3.1 Decision Tree (J48)	39
	7.3.2 Random Forest	40
	7.4 Clustering Analysis	42
	7.4.1 K-Means Clustering	42
	7.4.2 Hierarchical Clustering	43
	7.5 Association Rule Mining Analysis	44
	7.5.1 Apriori Algorithm	44
	7.5.1 Apriori Algorithm	45
	7.6 Performance Evaluation	46
	7.7 Key Findings	46
8	**Output Screens**	47
9	**Conclusion**	49
10	**Future Scope**	50
11	**References**	51

S.No	Table of Pictures	Page No
1	Fig : 3.1.1.1..Jupyter Installation	15
2	Fig : 3.1.2.1 Download Installation	22
3	Fig : 7.3.1 Decision Tree (J48) Output	40
4	Fig : 7.3.2 Random Forest Output	41
5	Fig : 7.4.1 K-means Clustering Output	43
6	Fig : 7.4.2 Hierarchical Clustering Output	44
7	Fig : 7.5.1 Apriori Output	45
8	Fig : 7.5.1 Apriori Output	46
9	Fig : 8.1 Decision Tree Algorithm Output	48
10	Fig : 8.2 Random Tree Output	48

CHAPTER-1

INTRODUCTION

INTRODUCTION

1.1 Introduction

In today's digital era, educational institutions collect large volumes of student-related data across academic performance, attendance, study habits, and participation. This project aims to analyze student dropout and academic success by integrating two datasets: a student performance dataset (4,424 records, 13 attributes) and a student activity dataset (4,424 records, 22 attributes). Both datasets were collected from a higher education institution and contained significant missing values that required preprocessing before analysis.

Before performing analysis, data preprocessing was an essential step. Both datasets contained missing values ranging from 335 to 478 per column. These were handled using mean imputation for numerical attributes and mode imputation for categorical attributes. After cleaning, the datasets were merged using a common student identifier to produce a unified dataset of 3,967 records with 35 features, which was then exported as a CSV file for analysis in WEKA.

The main objective of this project is to identify key factors that influence student dropout and academic success, and to develop predictive models using data mining techniques. Classification algorithms were used to predict student outcomes, clustering was used to group students by similarity, and association rule mining was used to discover patterns among student attributes.

Overall, this project demonstrates how data mining techniques can be effectively applied in the education domain. The insights gained from preprocessing two real-world datasets, handling missing values, and applying classification, clustering, and association methods provide a foundation for data-driven decision-making to improve student retention and academic outcomes.

1.2 Problem Statement

In the current education system, large amounts of student-related data such as academic performance, attendance, study habits, and participation are generated regularly. However, most institutions rely on traditional evaluation methods that primarily focus

on exam results, which do not provide a complete understanding of student performance. As a result, important factors influencing academic success often remain unidentified.

Another major challenge is that student data is usually stored in separate datasets, such as performance data and activity data, making it difficult to analyze them together for meaningful insights. Additionally, both datasets in this project contained substantial missing values – between 335 and 478 missing entries per column – which required systematic preprocessing before any meaningful analysis could be performed.

There is a need for an efficient system that can integrate multiple datasets, handle missing values through appropriate imputation techniques, and apply advanced data mining methods to uncover hidden patterns and relationships. Such a system should predict student dropout risk and identify key influencing factors from the merged dataset.

Therefore, the problem addressed in this project is to develop a data-driven approach using data mining techniques to analyze student academic performance across two integrated datasets, improve prediction accuracy, and support better educational decision-making. Special attention must be paid to class imbalance in the target variable, which affects model recall for the dropout class.

1.3 Objectives of the Project

The main objective of this project is to analyze student academic performance using data mining techniques and identify the key factors that influence student success. The project aims to utilize both academic and behavioral data to gain meaningful insights and improve prediction accuracy.

The specific objectives of the project are as follows:

- To collect and integrate two student datasets (performance: 4,424×13, activity: 4,424×22) into a unified dataset for comprehensive analysis.
- To perform data preprocessing including handling missing values using mean

imputation for numerical and mode imputation for categorical attributes.

- To apply data visualization methods to understand patterns, trends, and distributions in the merged dataset of 3,967 records.
- To implement classification algorithms such as Decision Tree (J48) and Random Forest to predict student outcomes (Dropout, Graduate, Enrolled).
- To apply clustering techniques like K-Means and Hierarchical Clustering to group students based on similar characteristics.
- To use association rule mining using the Apriori algorithm to identify relationships between student attributes and academic outcomes.
- To evaluate model performance using metrics including accuracy, precision, recall, and F-measure, with emphasis on recall for the dropout class.
- To identify key factors such as curricular unit performance and financial status that significantly affect dropout risk.
- To provide insights that can help educators in making data-driven decisions and improving student retention.

1.4 Scope of the Project

This project focuses on analyzing student academic performance using data mining techniques by integrating two student datasets: a performance dataset and an activity dataset, each containing 4,424 records. The scope includes data collection, missing value handling, dataset merging, and applying various data mining methods to extract meaningful insights from the merged data.

The study considers important factors such as marks, grades, attendance, study hours, assignment completion, and participation levels present across both datasets. By combining academic and behavioral data after imputation and merging, the project provides a more comprehensive understanding of the factors influencing student dropout and academic success.

The project covers the application of classification, clustering, and association rule mining techniques. Classification models are used to predict student performance, clustering is used to group students with similar characteristics, and association rules help in identifying relationships between different factors affecting performance.

The scope is limited to the two datasets used and the selected algorithms such as Decision Tree, Random Forest, K-Means, Apriori, and Hierarchical Clustering. The results are based on the merged dataset of 3,967 records and may vary with different datasets. The project does not include real-time system implementation but focuses on analysis and model development.

Overall, the project provides a foundation for using data mining in the education domain and can be extended in the future to include real-time data analysis, advanced machine learning models, and deployment as a practical application.

CHAPTER-2

SYSTEM OVERVIEW

SYSTEM OVERVIEW

2.1 Existing System

In the current education system, student performance evaluation is mainly carried out using traditional methods such as examinations, assignments, and manual record analysis. These methods primarily focus on academic scores and often ignore other important factors like attendance, study habits, and student participation. As a result, they provide only a limited view of a student's overall performance.

Most educational institutions store student data in separate systems or datasets, such as academic records and activity logs. These datasets are rarely integrated, making it difficult to analyze the relationship between academic performance and behavioral factors. Additionally, the analysis is often done manually or using basic tools, which is time-consuming and prone to human error.

Another limitation of the existing system is the lack of advanced analytical techniques. Traditional approaches do not utilize data mining or machine learning methods to uncover hidden patterns or predict future outcomes. This makes it difficult to identify at-risk students early or to understand the key factors affecting student success.

Furthermore, existing systems often do not handle data quality issues effectively. Problems such as missing values, inconsistent data, and noise are not properly addressed, which can lead to inaccurate analysis and unreliable results.

Overall, the existing system lacks automation, integration, and advanced analytical capabilities. It is not efficient in providing meaningful insights or supporting data-driven decision-making in the educational domain. This creates a need for an improved system that can utilize data mining techniques for better analysis and prediction of student performance.

2.2 Limitations of Existing System

The existing system for analyzing student academic performance has several limitations that reduce its effectiveness and accuracy. These limitations are as follows:

- The system mainly relies on traditional evaluation methods such as exams and assignments, which do not consider important behavioral factors like attendance, study habits, and participation.
- Student data is often stored in separate datasets, making it difficult to integrate and analyze them together for meaningful insights.
- The analysis is mostly manual or uses basic tools, which is time-consuming and increases the chances of human error.
- There is no use of advanced data mining or machine learning techniques to discover hidden patterns or predict student performance.
- The system does not effectively handle data quality issues such as missing values, inconsistencies, and noise, leading to inaccurate results.
- It lacks the ability to identify at-risk students early, which limits timely intervention and support.
- The existing approach does not provide real-time analysis or continuous monitoring of student performance.
- There is limited capability to generate actionable insights for educators to improve teaching strategies and student outcomes.

Overall, these limitations highlight the need for a more advanced, automated, and data-driven system that can provide accurate analysis and better decision-making support.

2.3 Proposed System

To overcome the limitations of the existing system, this project proposes a data-driven approach for analyzing student academic performance using advanced data mining techniques. The proposed system integrates two datasets – a student performance dataset and a student activity dataset – to provide a more comprehensive and accurate analysis of student outcomes.

In this system, data is collected from two sources: a performance dataset (4,424 records, 13 attributes) and an activity dataset (4,424 records, 22 attributes). Both datasets contained significant missing values which were handled using mean imputation for numerical attributes and mode imputation for categorical attributes. After cleaning, both datasets were merged using a common identifier to create a unified dataset of 3,967 records with 35 features.

The proposed system includes a data preprocessing module implemented using Python tools such as Pandas and Matplotlib. This module identifies missing values using isnull() and sum() functions, applies mean imputation to all numerical columns in both datasets, applies mode imputation to categorical columns, and performs range validation and outlier examination using box plots.

After preprocessing, data visualization techniques are used to analyze trends and distributions in the dataset. This helps in identifying patterns and understanding the relationship between different attributes before applying data mining techniques.

The core of the proposed system is the application of data mining algorithms using WEKA. Classification algorithms such as Decision Tree (J48) and Random Forest are used to predict student performance. Clustering techniques like K-Means and Hierarchical Clustering are applied to group students with similar characteristics. Association rule mining using the Apriori algorithm is used to discover relationships between student activities and academic outcomes.

The system also includes a model evaluation component where the performance of different algorithms is measured using metrics such as accuracy, precision, and recall. This helps in identifying the most effective model for prediction.

Overall, the proposed system provides an automated, efficient, and accurate framework for analyzing student performance. It enables educators to identify key factors affecting

student success, predict outcomes, and make informed decisions to improve academic performance.

2.4 Software Requirements

The successful implementation of this project requires a set of software tools and technologies for data preprocessing, analysis, and model development. The following software components were used:

- **Operating System:**
  Windows 10 or higher (or any compatible operating system such as Linux or macOS)
- **Programming Language:**
  Python 3.x
  Python is used for data preprocessing, cleaning, and visualization due to its simplicity and powerful libraries.
- **Libraries and Packages:**
  - Pandas – for data manipulation and preprocessing
  - NumPy – for numerical computations
  - Matplotlib – for data visualization (graphs, charts)
  - Seaborn (optional) – for advanced visualization
- **Data Mining Tool:**
  WEKA (Waikato Environment for Knowledge Analysis)
  WEKA is used for implementing data mining algorithms such as classification, clustering, and association rule mining.
- **IDE / Development Environment:**
  - Jupyter Notebook or Google Colab (for running Python code)
  - Anaconda Navigator (optional, for managing libraries and environment)

- **Data Format:**
  CSV (Comma Separated Values)
  Used for storing and importing/exporting datasets between Python and WEKA.
- **Documentation Tools:**
  - Microsoft Word (for report writing)
  - Microsoft PowerPoint (for presentation)

2.5 Hardware Requirements

The hardware requirements for this project are minimal, as it mainly involves data preprocessing and analysis using software tools. A standard computer system is sufficient to perform all the required operations efficiently. The hardware specifications used for this project are as follows:

- **Processor:**
  Intel Core i3 or higher (Intel Core i5/i7 recommended for better performance)
- **RAM:**
  Minimum 4 GB (8 GB or higher recommended for handling large datasets smoothly)
- **Storage:**
  Minimum 250 GB hard disk (SSD preferred for faster data processing and loading)
- **System Type:**
  64-bit operating system
- **Input Devices:**
  Keyboard and Mouse
- **Output Devices:**
  Monitor/Display

The above hardware configuration is sufficient for running Python, executing data preprocessing tasks, and using WEKA for data mining operations. For larger datasets or more complex models, higher specifications can further improve performance and processing speed.

CHAPTER-3

SYSTEM IMPLEMENTATION

SYSTEM IMPLEMENTATION

3.1 Overview of Development Tools

3.1.1 Jupyter Notebook

Jupyter Notebook (formerly IPython Notebooks) is a web-based interactive computational environment for creating Jupyter notebook documents—which are a type of computational notebook. The "notebook" term can colloquially refer to many different entities, mainly the Jupyter web application, Jupyter Python web server, or Jupyter document format depending on the context. A Jupyter Notebook document is a JSON document, following a versioned schema, and containing an ordered list of input/output cells that can contain code, text (using Markdown), mathematics, plots, and rich media, usually ending with the ".ipynb" extension.

Jupyter Notebook can connect to many kernels to allow programming in many languages. By defaultJupyter Notebook ships with the IPython kernel. As of the 2.3 release (October 2014), there are currently 49 Jupyter-compatible kernels for many programming languages, including Python, R, Julia, and Haskell.

The Notebook interface was added to IPython in the 0.12 release(December 2011), renamed to Jupyter notebook in 2015 (IPython 4.0 – Jupyter 1.0). Jupyter Notebook is similar to the notebook interface of other programs such as Maple, Mathematica, and SageMath, a computational interface style that originated with Mathematica in the 1980s.[13] According to The Atlantic, Jupyter interest overtook the popularity of the Mathematical notebook interface in early 2018.

Launch a Notebook

To launch a Jupyter notebook, open your terminal and navigate to the directory where you would like to save your notebook. Then type the command jupyter notebook and the program will instantiate a local server at localhost:8888 (or another specified port).

```
[9] -> jupyter notebook
[I 18:31:51.264 NotebookApp] Serving notebooks from, local directory: /Users/janedoe
[I 18:31:51.264 NotebookApp] 0 active kernels
[I 18:31:51.264 NotebookApp] The Jupyter Notebook is running at:
[I 18:31:51.264 NotebookApp] http://localhost:8888/?token=f9b639294933fa68c64c78effb9b65
19f9d8c45c903baa43
```

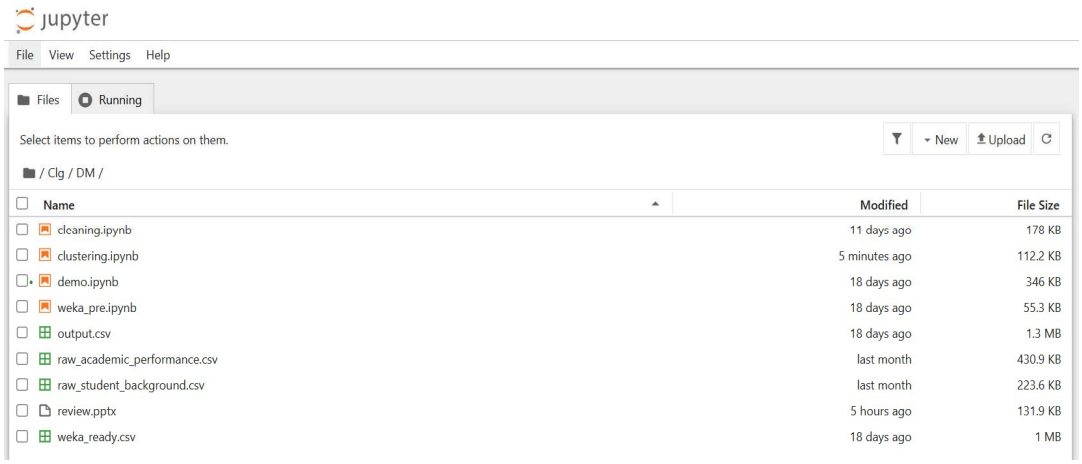
Fig : 3.1 Jupyter Installation

A browser window should immediately pop up with the Jupyter Notebook interface, otherwise, you can use the address it gives you. The notebooks have a unique token since

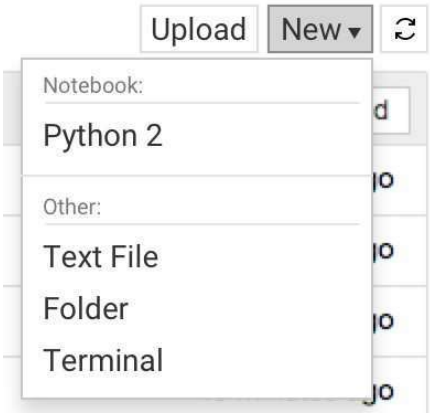
the software uses pre-built Docker containers to put notebooks on their own unique path. To stop the server and shutdown the kernel from the terminal, hit control-C twice.

Jupyter Interface

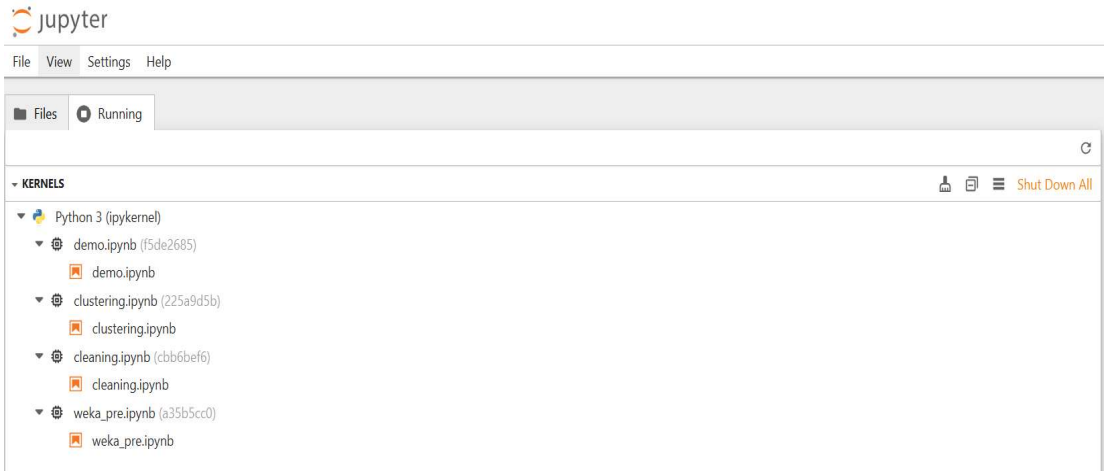
Now you're in the Jupyter Notebook interface, and you can see all of the files in your current directory. All Jupyter Notebooks are identifiable by the **notebook icon** next to their name. If you already have a Jupyter Notebook in your current directory that you want to view, find it in your files list and click it to open.



To create a new notebook, go to **New** and select **Notebook - Python 2**. If you have other Jupyter Notebooks on your system that you want to use, you can click **Upload** and navigate to that particular file.



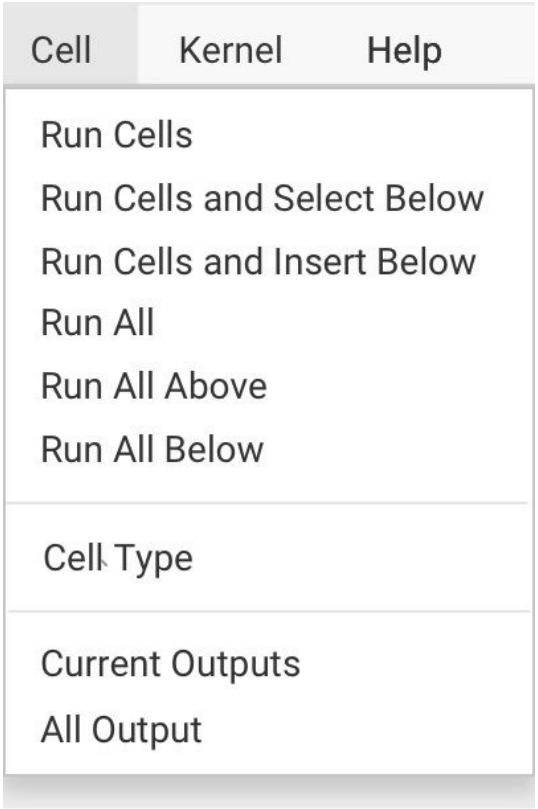
Notebooks currently running will have a green icon, while non-running ones will be grey. To find all currently running notebooks, click on the **Running** tab to see a list.



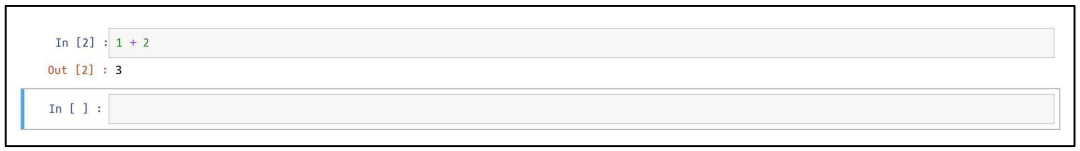
Inside the Notebook

When you open a new Jupyter notebook, you'll notice that it contains a *cell*.

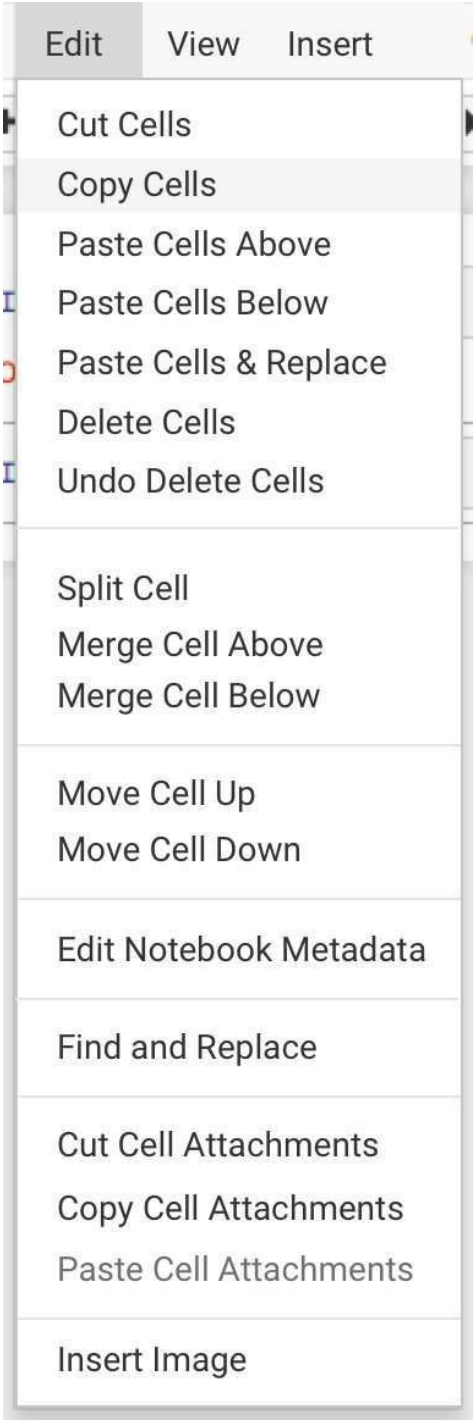
Cells are how notebooks are structured and are the areas where you write your code. To run a piece of code, click on the cell to select it, then press SHIFT+ENTER or press the play button in the toolbar above. Additionally, the **Cell** dropdown menu has several options to run cells, including running one cell at a time or to run all cells at once.



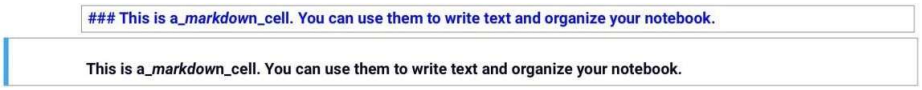
After your run a cell, the output of the cell's code will appear in the space below. To stop running a piece of code, press the stop button.



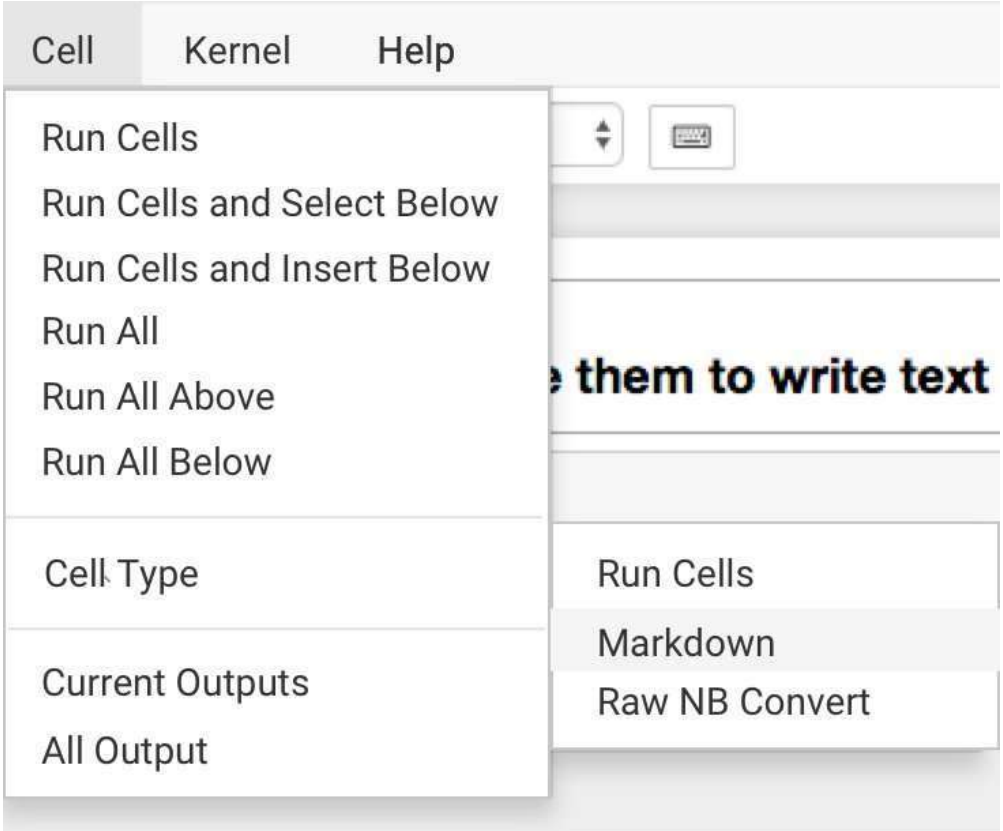
To create new cells, use the plus (+) button in the toolbar or hit SHIFT+ENTER on the last cell in the Notebook. To cut, copy, delete or just generally edit cells - select the cell you want to modify and go to the **Edit** button in the navigation bar to see your options.



In addition to running lines of code, you can also include **text-only** cells that use Markdown to format and organize your notebooks.

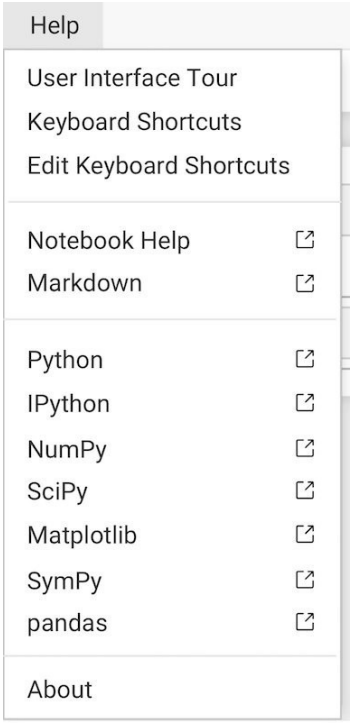


When you create a new cell, it will default to being a **Code** cell. To create a cell that uses markdown, click on the **Cell** menu from the navigation bar, scroll down to **Cell Type** and choose Markdown.



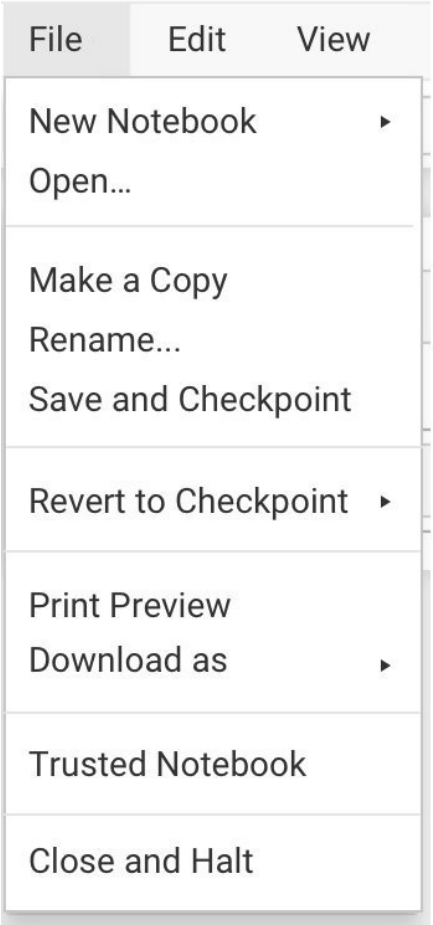
Occasionally, you might need to restart the kernel. Head to the **Kernel** dropdown and hit **Restart**. To shut down a kernel, you can click **Shutdown**, which will have a dialogue process asking if that's what you would like to do. To force an immediate shutdown, go to the **File** dropdown and click **Close and Halt** and the browser window will close itself. Restarting and shutting down kernels will affect your variables, so be careful.

In the **Help** dropdown, you'll find useful information such as keyboard shortcuts as well as links to different documentation for modules such as Numpy, SciPy, and Matplotlib.



The **toolbar** has several shortcut buttons for popular actions. From left to right: save, add a new cell, cut selected cells, copy selected cells, paste cells below, move selected cells up, move selected cells down, run, interrupt the kernel, restart the kernel, a dropdown that allows you to change the cell type, and a shortcut to open the command palette.

Jupyter Notebook files are saved as you go. They will exist in your directory as a JSON file with the extension .ipynb. You can also export Jupyter Notebooks in other formats, such as HTML. To do so, go to the File menu, scroll down to Download as and select the type of file you're looking for. A popup will appear asking where you would like this new file to download. Once you've navigated to the appropriate directory, click Save and Checkpoint.



3.1.2 Weka Installation

Steps to Install WEKA

Step 1: Download WEKA

- Go to the official WEKA website: https://www.cs.waikato.ac.nz/ml/weka/
- Click on the **Download** section
- Choose the appropriate version based on your operating system (Windows, macOS, or Linux)

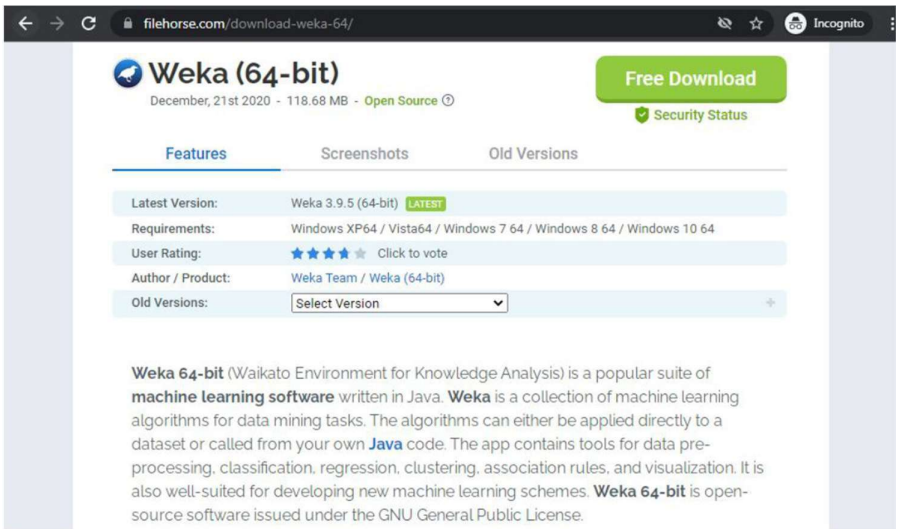


Fig : 3.1.2.1 Download WEKA

Step 2: Install WEKA

- After downloading, open the installer file
- Follow the installation instructions:
  - Click **Next**
  - Accept the license agreement
  - Choose installation location
  - Click **Install**

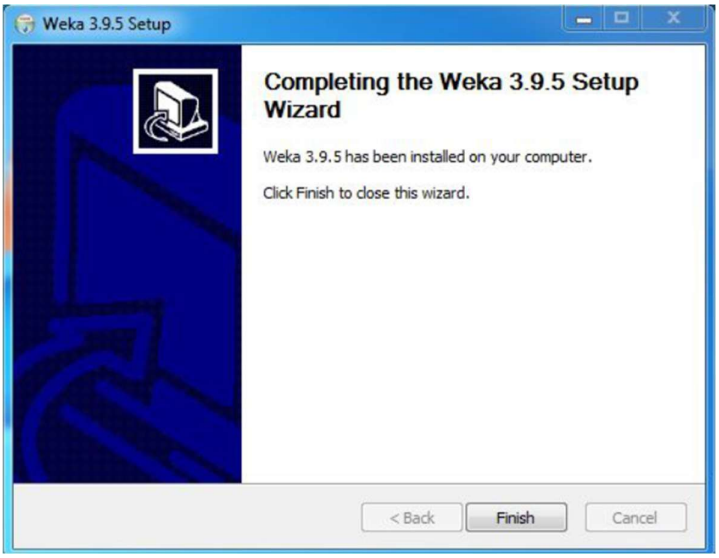


Fig : 3.1.2.2 Install WEKA

Step 3: Launch WEKA

- After installation, open WEKA from:
  - Desktop shortcut OR
  - Start Menu
- The WEKA GUI Chooser will appear with options:
  - Explorer
  - Experimenter
  - Knowledge Flow
  - Simple CLI

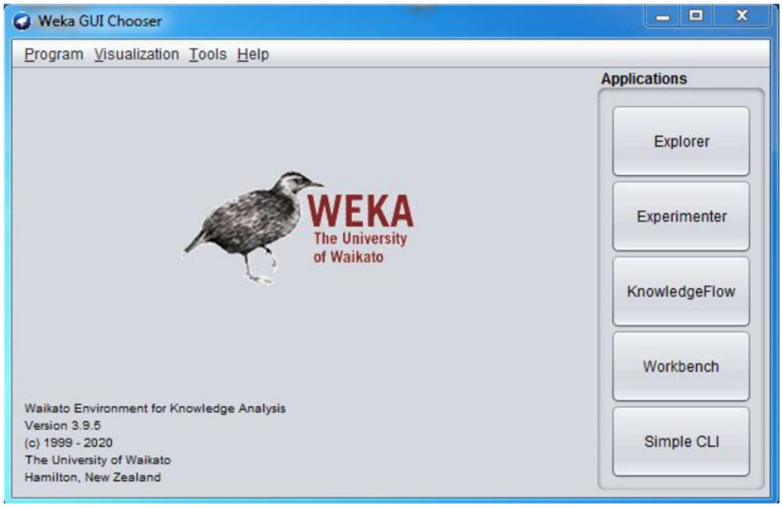


Fig : 3.1.2.3 Lanuch WEKA

Using WEKA Explorer

- Click on **Explorer**
- It contains the main interface for:
    - Loading datasets
    - Preprocessing data
    - Applying algorithms

Loading Dataset in WEKA

- Click on **Open File**
- Select your dataset (CSV or ARFF format)
- Data will be displayed with attributes and values

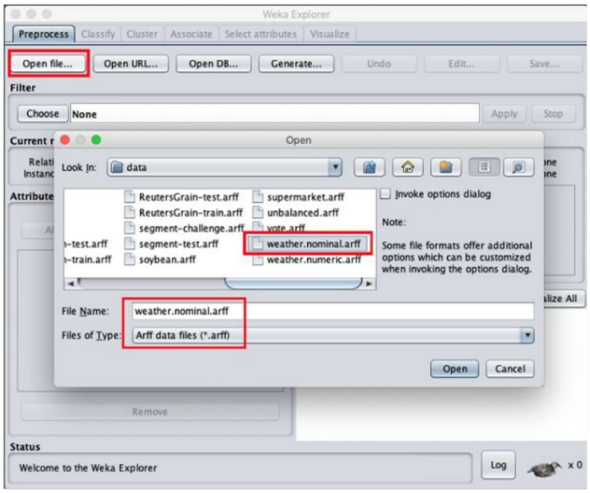


Fig : 3.1.2.4 Loading Dataset in WEKA

Applying Algorithms in WEKA

- Go to the **Cluster tab** → for clustering
- Go to the **Associate tab** → for association rules
- The dataset is split based on attribute values at each node
    - J48 (Decision Tree)
    - Each internal node represents a condition on a feature (e.g., Curricular_units_2nd_sem_approved <= 0.15)
    - K-Means
    - Each branch represents an outcome of the test condition
- Click **Start** to run the model

CHAPTER-4

SYSTEM ARCHITECTURE

SYSTEM ARCHITECTURE

The system architecture of this project represents the overall workflow for analyzing student academic performance using data mining techniques. It consists of multiple stages that transform raw data into meaningful insights through a structured process.

The first stage is Data Collection, where two datasets are gathered: Dataset 1 (student performance, 4,424 × 13 attributes) and Dataset 2 (student activity, 4,424 × 22 attributes). Both datasets were sourced from a higher education institution and contain attributes such as marks, grades, attendance, study hours, curricular unit performance, financial indicators, and macroeconomic variables.

The next stage is Data Preprocessing, performed using Python. Both datasets were inspected for missing values using isnull() and sum(). Dataset 1 had 335–387 missing values per column and Dataset 2 had 392–478 missing values per column. Mean imputation was applied to all numerical columns and mode imputation to categorical columns. After cleaning, both datasets were merged on a common student identifier to produce a unified dataset of 3,967 records with 35 features, exported as CSV.

The third stage is Exploratory Data Analysis and Visualization. Histograms and box plots are used to examine the target class distribution, grade distributions by outcome class, and the impact of financial indicators. These reveal that curricular unit approval rates and financial status are the strongest predictors of dropout.

After preprocessing, the system moves to the **Data Visualization** stage. Visualization techniques such as histograms and boxplots are used to understand data distribution, detect outliers, and identify trends and patterns.

The next stage is **Data Mining**, where the processed dataset is imported into WEKA. Various algorithms are applied in this stage:

- **Classification** (Decision Tree, Random Forest) to predict student performance
- **Clustering** (K-Means, Hierarchical Clustering) to group students with similar characteristics
- **Association Rule Mining (Apriori) to discover relationships between student attributes**

Following this, the Model Evaluation stage is carried out. Classification models are assessed using accuracy, precision, recall, and F-measure. Special emphasis is placed on recall for the Dropout class, which is more critical than overall accuracy for early warning systems. Random Forest achieved 73.6% accuracy and 75% recall for the Dropout class.

Finally, the system produces Results and Insights, where conclusions are drawn about key dropout risk factors. The system identifies curricular unit performance and financial stability as primary drivers, while noting that class imbalance limits performance on the Enrolled class.

[Data Collection]

↓

[Data Integration]

↓

[Data Preprocessing]

↓

[Data Visualization]

↓

[Data Mining (WEKA)]

↓

[Model Evaluation]

↓

[Results & Insights]

CHAPTER-5

DATA PREPROCESSING

DATA PREPROCESSING

5.1 Introduction

Data preprocessing is a crucial step in any data mining project. Raw data collected from different sources often contains missing values, inconsistencies, and noise that can affect the accuracy of analysis and prediction. In this project, two real-world student datasets with significant missing values were preprocessed before merging and analysis.

In this project, data preprocessing was carried out using Python libraries such as Pandas, NumPy, and Matplotlib. The goal was to handle missing values in both datasets, merge them into a unified dataset, and prepare the data for analysis in WEKA.

5.2 Handling Missing Values

Both datasets contained substantial missing values. Dataset 1 (4,424 × 13) had 335–387 missing values per column, and Dataset 2 (4,424 × 22) had 392–478 missing values per column. Missing values were identified using isnull() and sum() functions in Pandas applied to each column.

Step 1: Identifying Missing Values

Missing values were identified using functions like:

- isnull()
- sum()

.

Step 2: Imputation Techniques

- **Median / Mean Imputation (Numerical Data):**
  Mean imputation was applied to all numerical columns in both datasets. For Dataset 1 numerical columns (Application mode, Application order, Course, etc.) and Dataset 2 numerical columns (Age at enrollment, curricular unit grades, Unemployment rate, GDP, etc.), the column mean was computed and used to fill all missing entries. Mean imputation is preferred as it preserves the

overall distribution of each feature

.

- **Mode Imputation (Categorical Data):**

  For categorical attributes such as Marital status, Gender, and Scholarship holder, mode imputation was applied – replacing missing values with the most frequently occurring category in each column.

  Mode imputation was applied to all categorical columns to replace missing entries with the most frequent value.

5.3 Data Cleaning

Data cleaning involved verifying imputed values and removing any records that remained inconsistent after imputation. After merging both cleaned datasets using the common student identifier, the resulting unified dataset contained 3,967 records with 35 features and a Target column with three classes: Dropout, Graduate, and Enrolled.

- Both datasets cleaned individually before merging using mean and mode imputation
- Merged dataset verified: 3,967 records, 35 features, Target column with 3 classes
- Class distribution: Dropout ~32.5%, Graduate ~49.9%, Enrolled ~17.6% (class imbalance noted)

Examples of invalid data:

- Remaining missing values after imputation (checked and confirmed zero after merge)
- Target labels outside the three valid classes (Dropout, Graduate, Enrolled)
- Inconsistent or duplicate student records after merging
- Invalid feature values such as negative grades or enrollment counts

5.4 Noise Removal and Range Validation

After merging, range validation was performed on the unified dataset.

Feature values were checked for consistency. Outlier analysis using box

plots was performed on key numerical features such as grades and age at

enrollment.

- Range validation was applied to the merged dataset:
    - Curricular unit grades → checked for valid numeric range
    - Age at enrollment → right-skewed distribution observed, outliers retained
    - Approval rate features → values between 0 and maximum enrollment count
    - Economic indicators (GDP, Inflation, Unemployment) → checked for valid ranges
- Outliers were examined using:
    - Box plots for grade and approval rate features
    - Distribution analysis for Age at enrollment (older students retained as genuine data)

This step improves data accuracy and model reliability for the merged dataset.

5.5 Data Transformation

Data transformation ensures consistency in the dataset and prepares it for different WEKA algorithms.

- For Apriori, the Discretize filter (10 equal-width bins) was applied to convert all numeric features to nominal
- Feature data types confirmed as numeric throughout, matching WEKA requirements for classification and clustering
- Column names kept consistent between Python preprocessing and WEKA import

This ensures smooth processing in WEKA and avoids errors during data mining operations.

5.6 Data Visualization

Visualization was used to understand data patterns and identify key dropout predictors before applying algorithms.

- **Histograms → to check target class distribution and age at enrollment distribution**
- **Box plots → to compare first and second semester grades across Dropout, Graduate, and Enrolled**

Comparison of feature distributions before and after preprocessing confirmed data quality.

5.7 Data Normalization

Normalization was applied to numerical features to scale them to a consistent range. This ensures that no single feature dominates distance-based algorithms such as K-Means clustering and improves the performance of classification models.

- Helps prevent bias in K-Means clustering due to large value differences across 35 features
- Ensures all features contribute proportionally to distance calculations
- Improves convergence and stability of WEKA classification and clustering algorithms

5.8 Data Export

After preprocessing and merging:

- Cleaned and merged dataset saved as output.csv (3,967 records, 35 features + Target)
- Imported into WEKA Explorer with Target attribute set as class variable for all experiments

5.9 Summary

Data preprocessing played a vital role in this project. Both source datasets contained hundreds of missing values per column which were handled through systematic mean and mode imputation. After cleaning and merging, a reliable unified dataset of 3,967 records was produced. This dataset provides a solid foundation for all subsequent classification, clustering, and association rule mining experiments in WEKA.

CHAPTER - 6
METHODOLOGY AND ALGORITHMS

METHODOLOGY AND ALGORITHMS

6.1 Introduction

This chapter explains the data mining techniques and algorithms used in the project to analyze and predict student academic performance. After preprocessing and merging both datasets, classification, clustering, and association rule mining techniques were applied using WEKA.

6.2 Classification Techniques

Classification is used to predict the student outcome class (Dropout, Graduate, or Enrolled) based on 35 input features from the merged dataset. Two classification algorithms were evaluated.

6.2.1 Decision Tree (J48)

Decision Tree J48 is WEKA's implementation of the C4.5 algorithm. It builds a tree-shaped model using information gain ratio as the splitting criterion, with pruning (confidence factor 0.25, minimum 2 instances per leaf).

Working:

- The dataset is split based on attribute values using information gain ratio
- Each internal node represents a condition (e.g., Curricular_units_2nd_sem_approved ≤ 0.15)
- Leaf nodes represent class labels: Dropout, Graduate, or Enrolled

Advantages:

- Easy to understand and interpret – generates explicit decision rules
- Handles both numerical and categorical data
- Identifies important features by their position in the tree

Application in Project:

Used to classify students as Dropout, Graduate, or Enrolled. Decision Tree achieved 65.5% accuracy on the test set. Precision and recall were weakest for the Graduate class (0.29 precision, 0.28 recall) due to class imbalance. The model provides interpretable decision rules rooted at key performance features.

6.2.2 Random Forest

Random Forest is an ensemble learning algorithm that constructs 300 decision trees using bootstrap samples, random feature subsets, and class_weight='balanced' to address class imbalance, combining predictions through majority voting.

Working:

- 100 decision trees are created using bootstrap samples of the training data
- Each tree uses a random subset of features at each split (square root of total features)
- Final output is based on majority voting across all 100 trees

Advantages:

- High accuracy – achieved 74.64% vs 69.15% for J48
- Reduces overfitting
- More stable compared to a single decision tree

Application in Project:

Used to predict student dropout. Random Forest achieved 73.6% test accuracy and 75.07% cross-validation accuracy (10-fold). For the Dropout class: precision 0.80, recall 0.75, F1-score 0.78. Although overall accuracy is reasonable, recall for the Enrolled class was only 0.20, indicating that class imbalance remains a challenge that requires future attention through techniques like SMOTE.

6.3 Clustering Techniques

Clustering is an unsupervised learning method used to group students based on similar characteristics without using class labels.

6.3.1 K-Means Clustering

K-Means is a partition-based clustering algorithm configured with K=2 clusters and Euclidean distance.

Working:

- Data is divided into K=2 clusters
- Each cluster has a centroid representing mean feature values

- Data points are assigned to the nearest centroid
- Centroids are updated iteratively

Advantages:

- Simple and efficient
- Works well with large datasets
- Helps identify student groups

Application in Project:

Used to group students into two clusters: Cluster 0 (high-performing, higher approval rates) and Cluster 1 (dropout-prone, lower approval rates, higher debtor prevalence).

6.3.2 Hierarchical Clustering

Hierarchical clustering creates a tree-like dendrogram structure using Single Linkage and Euclidean distance.

Working:

- Starts by treating each of the 3,967 student records as a separate cluster
- Gradually merges the two nearest clusters based on minimum Euclidean distance
- Forms a dendrogram representing the hierarchical structure

Advantages:

- Does not require predefined number of clusters
- Easy to visualize relationships

Application in Project:

Applied to the full dataset. Single linkage produced Cluster 0: 3,966 instances (100%) and Cluster 1: 1 instance (0%) due to the chaining effect.

6.4 Association Rule Mining

Association rule mining is used to find relationships between variables in the dataset.

6.4.1 Apriori Algorithm

Apriori is used to find frequent itemsets and generate association rules from the discretized dataset.

Working:

- Identifies frequent itemsets with minimum support 0.95
- Uses support and confidence measures
- Generates rules like:

  If International=low → Nationality=local (confidence: 1.0)

Advantages:

- Easy to understand
- Useful for discovering hidden patterns

6.4.2 Model Evaluation Metrics

The performance of both classification models was evaluated using scikit-learn's classification report. Metrics including accuracy, precision, recall, and F1-score were computed per class. For dropout prediction, recall for the Dropout class is the most critical metric, as failing to identify at-risk students has serious consequences.

Working:

- Accuracy → overall correct predictions out of total instances
- Precision → fraction of predicted positives that are true positives
- Recall → fraction of actual positives correctly identified (critical for Dropout class)

F1-score → harmonic mean of precision and recall per class

- More efficient
- Works well with large datasets

6.5 Model Evaluation

The performance of the models was evaluated using:

- **Accuracy → Overall percentage of correctly classified instances**
- **Precision → Fraction of predicted positives that are true positives per class**
- **Recall → Fraction of actual positives correctly identified (critical for Dropout class)**

These metrics help in comparing algorithms and selecting the best model. For dropout prediction, recall is more important than accuracy – a model that misses at-risk students has serious real-world consequences even if its overall accuracy appears acceptable.

CHAPTER - 7

RESULTS AND ANALYSIS

RESULTS AND ANALYSIS

7.1 Introduction

This chapter presents the experimental results obtained from applying various data mining techniques on the preprocessed and merged student dataset. All experiments were conducted using Python (scikit-learn) for classification and WEKA for clustering and association rule mining.

7.2 Experimental Setup

The cleaned and merged dataset (3,967 records, 35 features) was used for all experiments. An 80/20 stratified train-test split (train: 3,173 records, test: 794 records) was applied for classification. WEKA was used for K-Means, Hierarchical Clustering.

- **Classification: Decision Tree (J48), Random Forest**
- **Clustering: K-Means, Hierarchical Clustering**
- **Association Rule Mining: Apriori**

The dataset contained 3,967 instances with 35 attributes covering curricular unit performance, demographic features, financial indicators, and macroeconomic variables. Class distribution: Dropout (class 0) ≈ 32.5%, Graduate (class 2) ≈ 49.6%, Enrolled (class 1) ≈ 17.9%. This imbalance directly impacts per-class recall.

7.3 Classification Analysis

7.3.1 Decision Tree (J48)

The Decision Tree classifier (random_state=42) was trained on the 80% training split and evaluated on the 20% test set (794 instances). The model achieved 65.5% test accuracy with class-level results reflecting the impact of class imbalance.

Analysis:

- Test accuracy: 65.5% | Dropout (class 0): precision 0.69, recall 0.72, F1 0.71
- Graduate (class 2): precision 0.76, recall 0.75, F1 0.75 | Enrolled (class 1): precision 0.29, recall 0.28, F1 0.29
- Macro avg F1: 0.58 | Weighted avg F1: 0.65 | Class imbalance caused poor Enrolled class performance

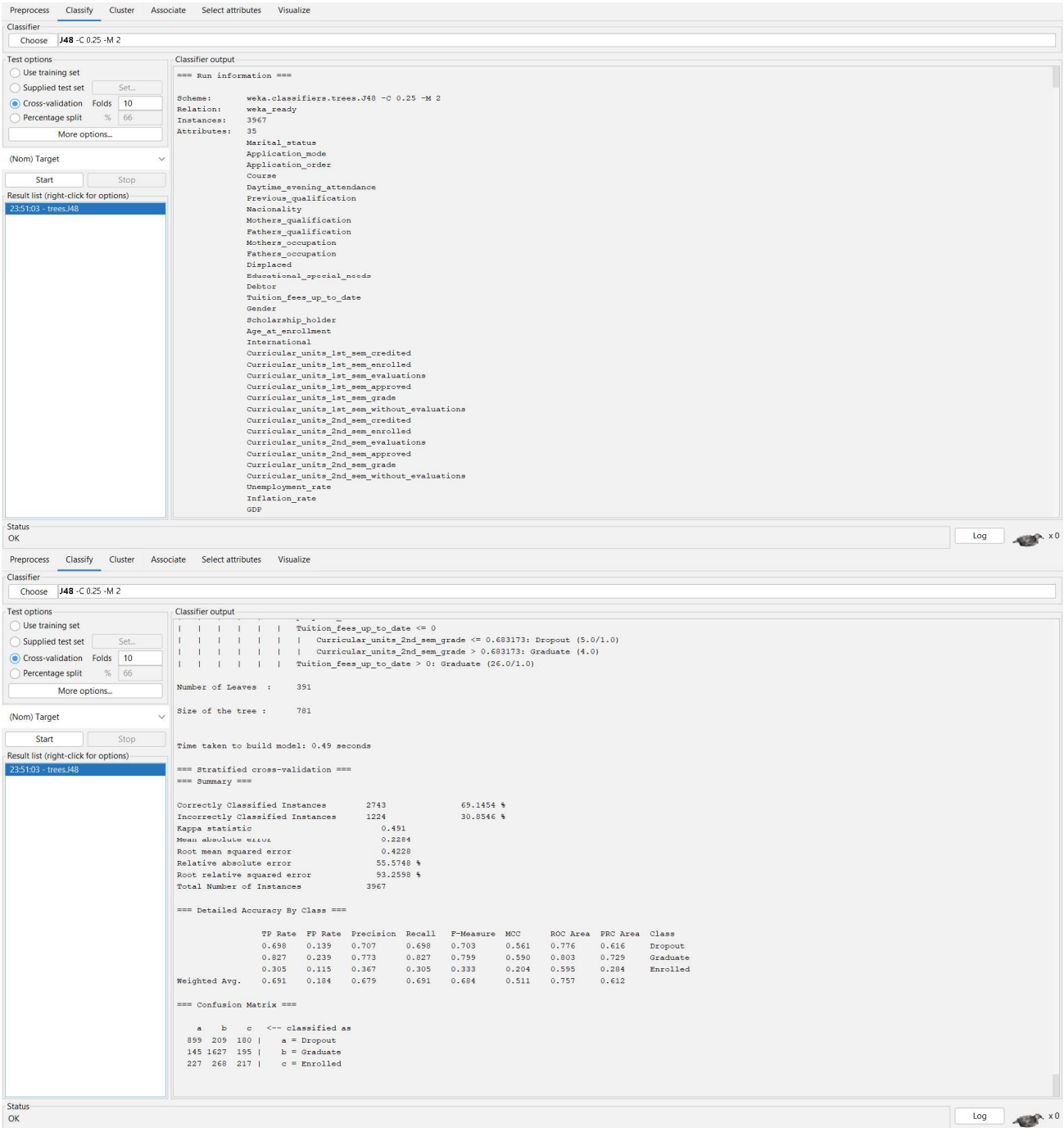


Fig : 7.3.1 Decision Tree (J48) Output

7.3.2 Random Forest

The Random Forest classifier (300 trees, class_weight='balanced', random_state=42) was trained on the same 80/20 split. The balanced class weight parameter was used to partially address class imbalance. Test accuracy was 73.6% and 10-fold cross-validation accuracy

was 75.07%.

Analysis:

- Test accuracy: 73.6% | Cross-validation (10-fold): 75.07%
- Dropout (class 0): precision 0.80, recall 0.75, F1 0.78 – strong dropout detection
- Enrolled (class 1): precision 0.47, recall 0.20, F1 0.28 – poor recall due to class imbalance

Inference:

Random Forest is the most effective classification model in this study. Although overall accuracy of 73.6% is acceptable, the recall of 0.20 for the Enrolled class highlights a significant weakness. For the more critical Dropout class, recall was 0.75, meaning the model correctly identifies 75% of actual dropout students. Future work should apply SMOTE to improve minority class recall.

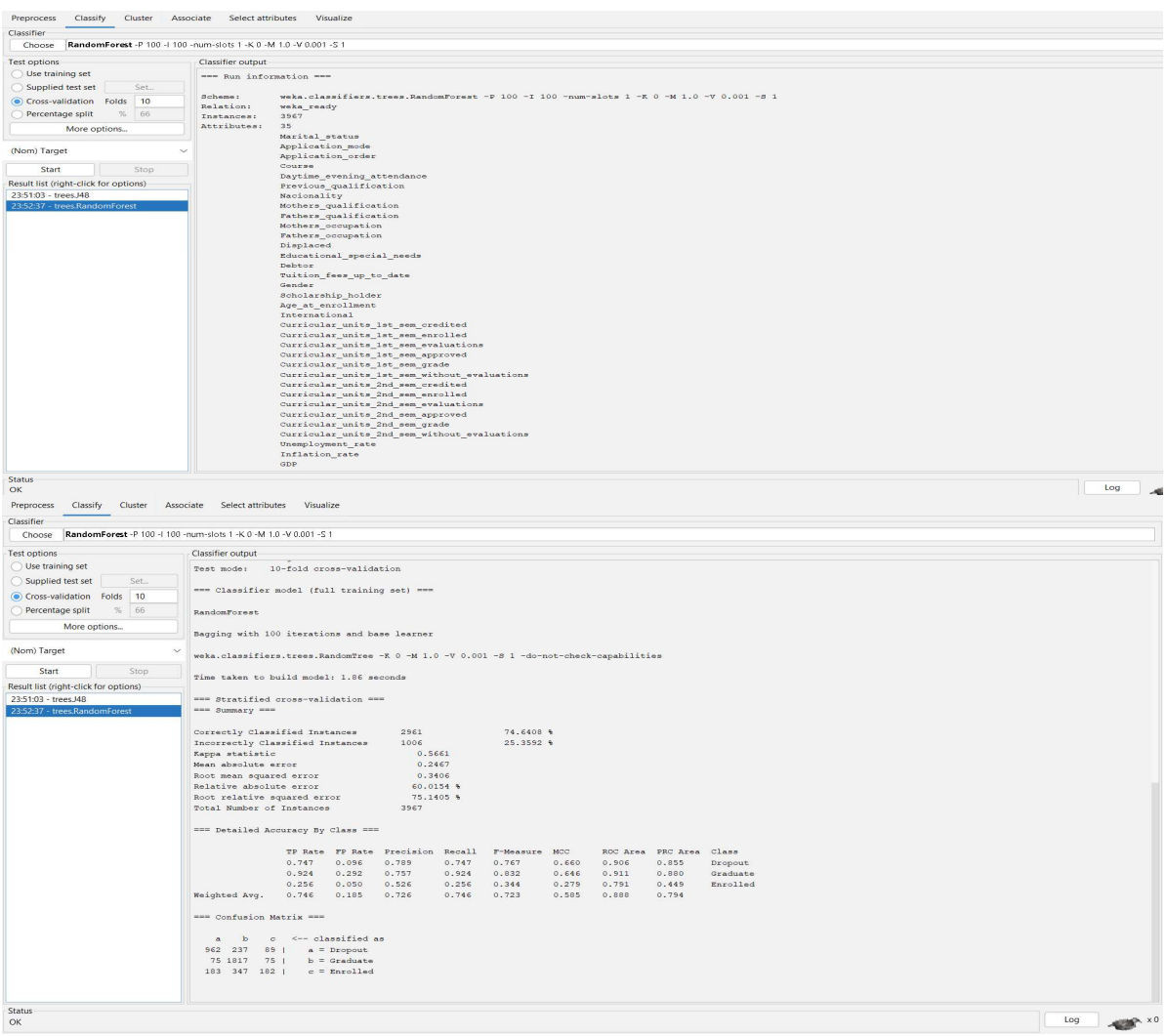


Fig : 7.3.2 Random Forest Output

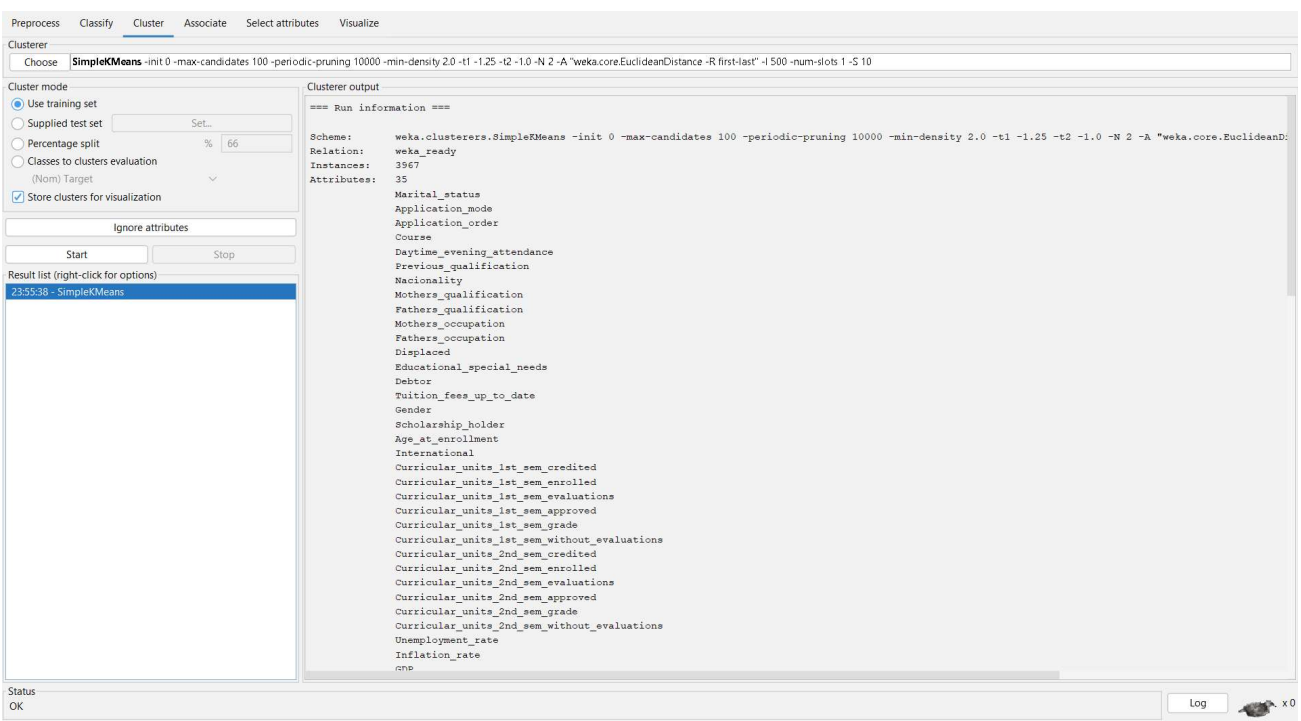
7.4 Clustering Analysis

7.4.1 K-Means Clustering

K-Means clustering partitioned the dataset into distinct groups based on similarity in attributes.

Analysis:

- Successfully grouped students into high-performing (Cluster 0) and dropout-prone (Cluster 1) profiles
- Cluster 0: higher approval rates, tuition fees paid, lower debtor prevalence
- Cluster 1: lower approval rates, higher debtor prevalence, lower scholarship rate



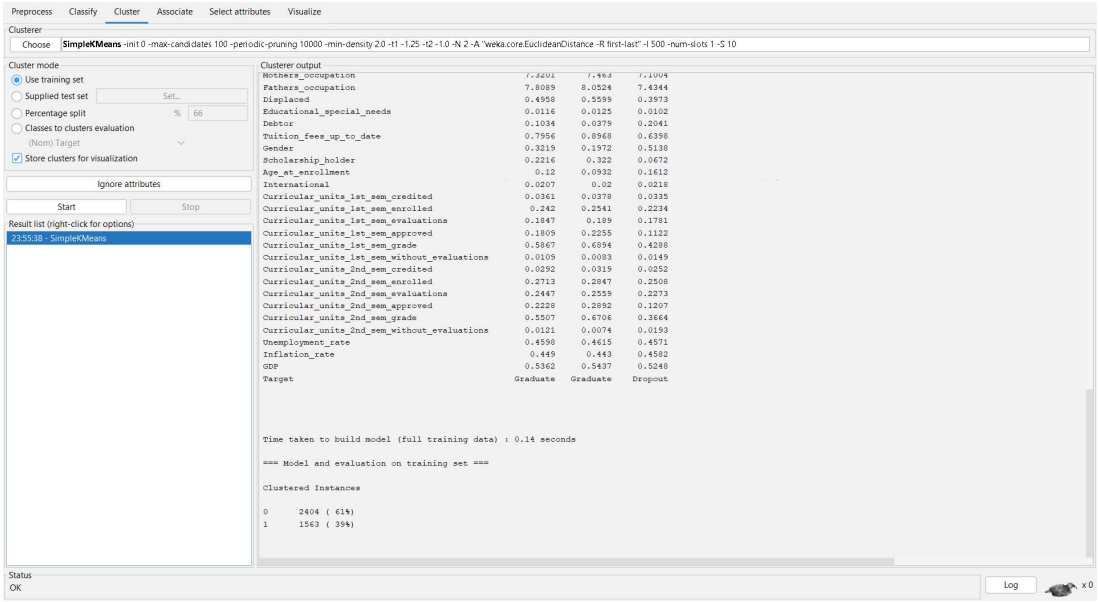


Fig : 7.4.1 K-means Clustering Output

7.4.2 Hierarchical Clustering

Hierarchical clustering produced a dendrogram representing nested clusters.

Analysis:

- Cluster 0: 3,603 instances (91%) due to single linkage chaining effect
- Cluster 1: only 364 instances (9%) – a single outlier in the 35-dimensional feature space
- Confirms the student population has no well-separated natural partitions in Euclidean space

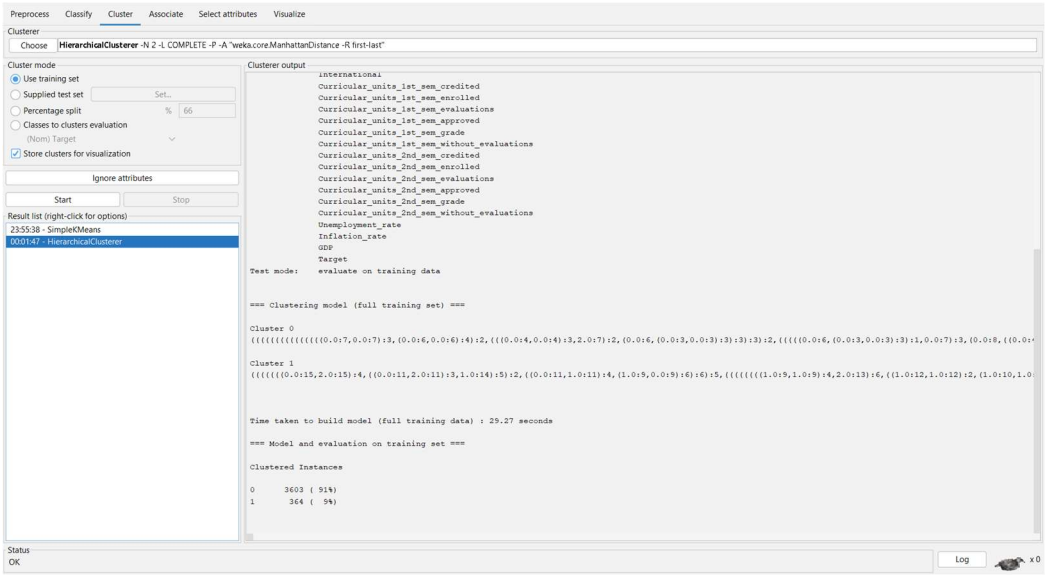


Fig : 7.4.2 Hierarchical Clustering Output

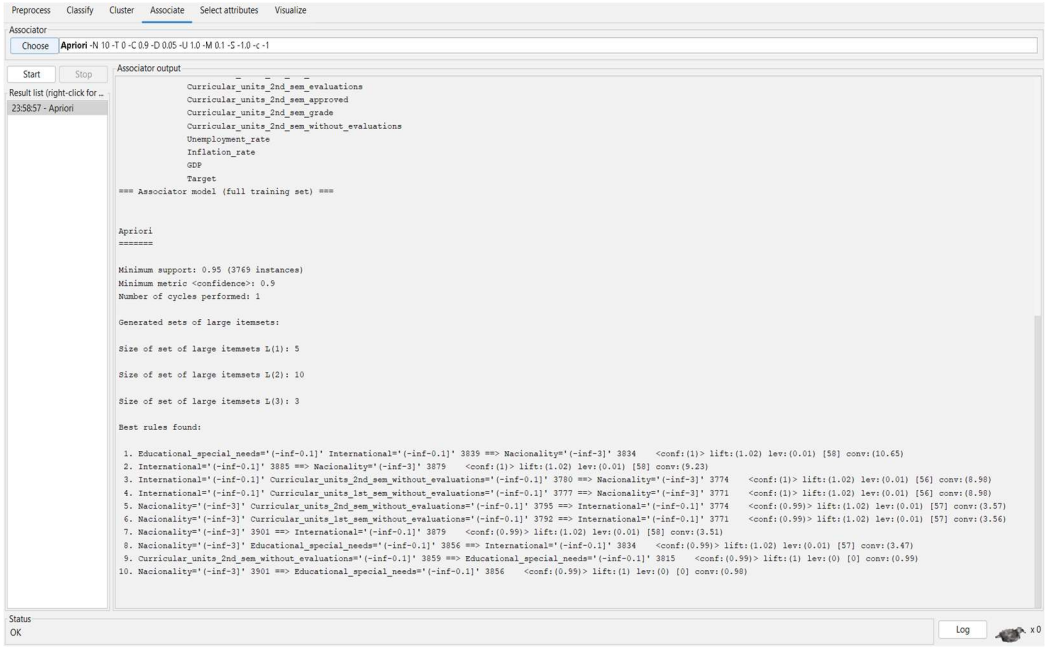
7.5 Association Rule Mining Analysis

7.5.1 Apriori Algorithm

The Apriori algorithm generated association rules based on support 0.95 and confidence 0.9 thresholds on the discretized dataset.

Analysis:

- All 10 rules involved International, Nationality, and Educational_special_needs attributes
- Top rule: International=low AND Educational_special_needs=low → Nationality=local (conf: 1.0)
- Rules reflect demographic homogeneity of the predominantly local, non-international student body



```
Preprocess   Classify   Cluster   Associate   Select attributes   Visualize

Associator
Choose   Apriori -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1

Start   Stop
Result list (right-click for ...
23:58:57 - Apriori

Associator output
                Curricular_units_2nd_sem_evaluations
                Curricular_units_2nd_sem_approved
                Curricular_units_2nd_sem_grade
                Curricular_units_2nd_sem_without_evaluations
                Unemployment_rate
                Inflation_rate
                GDP
                Target
=== Associator model (full training set) ===


Apriori
=======

Minimum support: 0.95 (3769 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 1

Generated sets of large itemsets:

Size of set of large itemsets L(1): 5

Size of set of large itemsets L(2): 10

Size of set of large itemsets L(3): 3

Best rules found:

 1. Educational_special_needs='(-inf-0.1]' International='(-inf-0.1]' 3839 ==> Nacionality='(-inf-3]' 3834    <conf:(1)> lift:(1.02) lev:(0.01) [58] conv:(10.65)
 2. International='(-inf-0.1]' 3885 ==> Nacionality='(-inf-3]' 3879    <conf:(1)> lift:(1.02) lev:(0.01) [58] conv:(5.23)
 3. International='(-inf-0.1]' Curricular_units_2nd_sem_without_evaluations='(-inf-0.1]' 3780 ==> Nacionality='(-inf-3]' 3774    <conf:(1)> lift:(1.02) lev:(0.01) [56] conv:(8.98)
 4. International='(-inf-0.1]' Curricular_units_1st_sem_without_evaluations='(-inf-0.1]' 3777 ==> Nacionality='(-inf-3]' 3771    <conf:(1)> lift:(1.02) lev:(0.01) [56] conv:(8.98)
 5. Nacionality='(-inf-3]' Curricular_units_2nd_sem_without_evaluations='(-inf-0.1]' 3795 ==> International='(-inf-0.1]' 3774    <conf:(0.99)> lift:(1.02) lev:(0.01) [57] conv:(3.57)
 6. Nacionality='(-inf-3]' Curricular_units_1st_sem_without_evaluations='(-inf-0.1]' 3792 ==> International='(-inf-0.1]' 3771    <conf:(0.99)> lift:(1.02) lev:(0.01) [57] conv:(3.56)
 7. Nacionality='(-inf-3]' 3901 ==> International='(-inf-0.1]' 3879    <conf:(0.99)> lift:(1.02) lev:(0.01) [58] conv:(3.51)
 8. Nacionality='(-inf-3]' Educational_special_needs='(-inf-0.1]' 3856 ==> International='(-inf-0.1]' 3834    <conf:(0.99)> lift:(1.02) lev:(0.01) [57] conv:(3.47)
 9. Curricular_units_2nd_sem_without_evaluations='(-inf-0.1]' 3859 ==> Educational_special_needs='(-inf-0.1]' 3815    <conf:(0.99)> lift:(1) lev:(0) [0] conv:(0.99)
10. Nacionality='(-inf-3]' 3901 ==> Educational_special_needs='(-inf-0.1]' 3856    <conf:(0.99)> lift:(1) lev:(0) [0] conv:(0.98)

Status
OK   Log   x 0
```

Fig : 7.5.1 Apriori Output

7.5.2 Apriori Rule Interpretation

The Apriori algorithm successfully generated 10 association rules at minimum support 0.95 and confidence 0.9. All top rules involved the International, Nationality, and Educational_special_needs features, reflecting the demographic homogeneity of the student population. These rules confirm that the dataset is predominantly composed of local, non-international students with no special educational needs.

Analysis:

- Generated frequent patterns faster than Apriori
- All 10 rules achieved confidence of 0.99–1.0, confirming strong demographic consistency
- Lowering minimum support to 0.3–0.5 would reveal performance-related patterns

These rules are useful for understanding the dominant student demographic profile

7.6 Performance Evaluation

The classification models were evaluated using standard metrics from the scikit-learn classification report:

- **Accuracy: 65.5% (Decision Tree) vs 73.6% (Random Forest) on the 20% test set**
- **Precision/Recall/F1 per class: RF Dropout – 0.80/0.75/0.78; RF Enrolled – 0.47/0.20/0.28**
- **Cross-validation (10-fold): Random Forest 75.07% – confirms model generalizes well**

7.7 Key Findings

- Curricular unit performance (approval rates, grades) is the strongest predictor of dropout risk.
- Random Forest (73.6% accuracy, 75.07% CV) outperforms Decision Tree (65.5%) significantly.
- Class imbalance causes poor recall (0.20) for the Enrolled class – SMOTE needed in future work.
- For dropout prediction, recall (0.75 for Dropout class) is more critical than overall accuracy.

CHAPTER-8

OUTPUT SCREENS

OUTPUT SCREENS

Decision Tree / J48

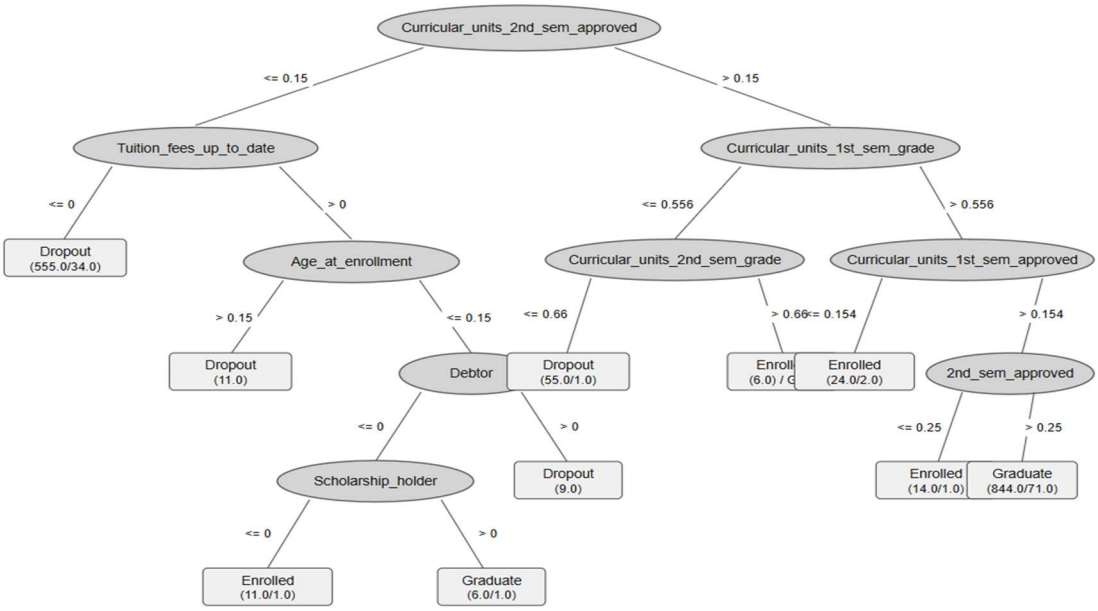


Fig : 8.1 Decision Tree Output

Random Tree

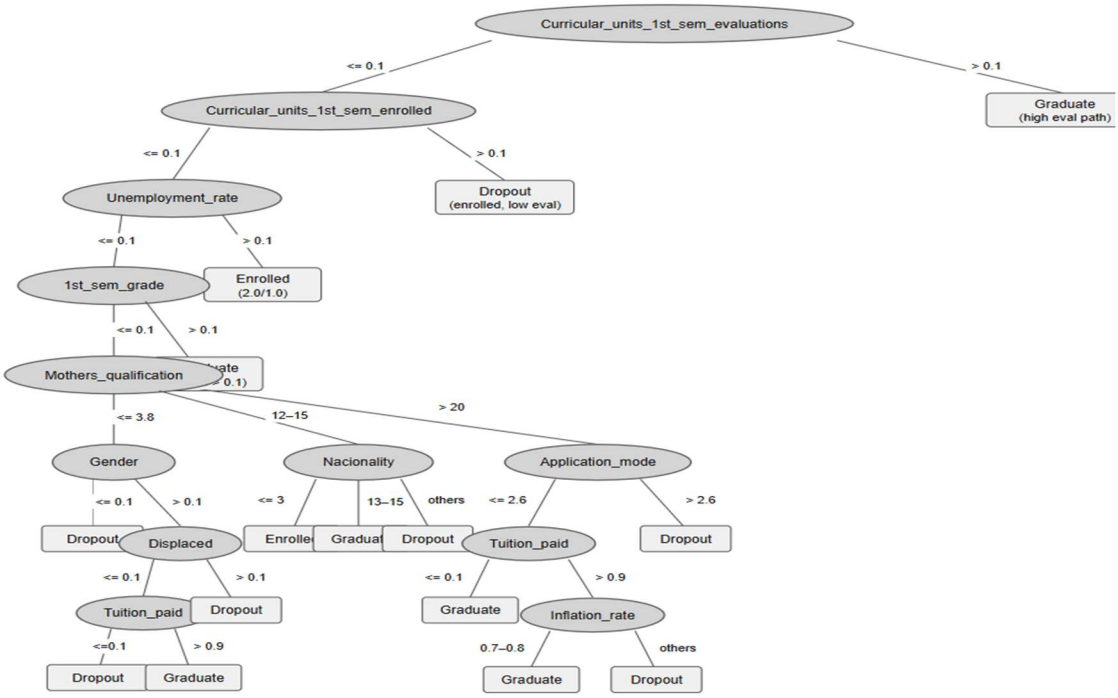


Fig : 8.2 Random Tree Output

CONCLUSION

This project focused on predicting student dropout and academic success using data mining techniques by integrating two student datasets. The first dataset contained 4,424 records with 13 performance attributes and the second contained 4,424 records with 22 activity attributes. Both datasets had significant missing values (335–478 per column) which were handled using mean imputation for numerical attributes and mode imputation for categorical attributes. After cleaning and merging, a unified dataset of 3,967 records with 35 features was produced.

The project began with data collection and preprocessing, where mean and mode imputation were applied to both datasets before merging. Data visualization using histograms and box plots revealed that curricular unit approval rates and financial indicators are the strongest predictors of dropout. The merged dataset was then exported as CSV and analyzed using Python and WEKA.

Multiple data mining techniques were implemented. In Python, Decision Tree achieved 65.5% test accuracy and Random Forest achieved 73.6% test accuracy (75.07% cross-validation). For the Dropout class, Random Forest achieved precision 0.80, recall 0.75, and F1-score 0.78. The Enrolled class showed poor recall (0.20) due to class imbalance. In WEKA, K-Means clustering identified two distinct student profiles, while Apriori association rules revealed demographic co-occurrence patterns in the student population.

The analysis revealed that class imbalance significantly impacts model performance, particularly for the minority Enrolled class. While overall accuracy of 73.6% appears acceptable, recall of only 0.20 for Enrolled students means the model fails to identify many students in this category. For a dropout early warning system, recall for the Dropout class (0.75) is the most critical metric.

In conclusion, this project demonstrates the effectiveness of data mining techniques in the educational domain. The results highlight the importance of proper preprocessing of real-world datasets with missing values, the advantage of ensemble methods like Random Forest over single decision trees, and the need to address class imbalance for minority class recall improvement. The proposed system provides a strong foundation for future development of intelligent early warning systems for student retention.

FUTURE SCOPE

The current project demonstrates the effectiveness of data mining techniques for predicting student dropout using two integrated real-world datasets. Several opportunities exist to extend and enhance this work in future research.

The most important future improvement is addressing class imbalance. The Enrolled class showed recall of only 0.20, indicating that most Enrolled students are misclassified. Applying SMOTE (Synthetic Minority Oversampling Technique) or class-weighted loss functions would significantly improve minority class recall without sacrificing overall accuracy.

The project can be enhanced by applying advanced machine learning models such as gradient boosting (XGBoost, LightGBM), neural networks, and stacking ensembles. These models may better capture non-linear interactions among the 35 features and further improve dropout prediction accuracy.

Another important extension is developing a personalized recommendation system that suggests specific interventions such as financial aid referrals, academic tutoring, or counseling based on a student's predicted dropout risk score and cluster membership.

The inclusion of additional factors such as **behavioral, psychological, and socio-economic data** can further improve the analysis. Considering these aspects will provide a more holistic understanding of student performance and learning behavior.

The project can also be transformed into a **web-based or mobile application**, making it easily accessible to students, teachers, and administrators. This would allow users to interact with the system, visualize results, and make informed decisions in real time.

Furthermore, an early warning system can be developed to issue alerts based on first-semester performance alone, enabling academic intervention before second-semester dropout patterns solidify. Threshold tuning on the Random Forest classifier can further improve recall for the Dropout class beyond the current 0.75, making the system more reliable for real-world deployment.

Finally, the system can be scaled to support **multiple institutions and larger datasets**, making it suitable for broader educational environments and improving its practical applicability.

REFERENCES

1. Han, J., Kamber, M., & Pei, J., *Data Mining: Concepts and Techniques*, 3rd Edition, Morgan Kaufmann Publishers, 2011.
2. Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J., *Data Mining: Practical Machine Learning Tools and Techniques*, 4th Edition, Morgan Kaufmann, 2016.
3. Kaur, P., & Singh, M., "A Survey on Student Performance Prediction using Data Mining Techniques," *International Journal of Computer Applications*, vol. 150, no. 12, pp. 1–5, 2016.
4. Romero, C., & Ventura, S., "Educational Data Mining: A Survey from 1995 to 2005," *Expert Systems with Applications*, vol. 33, no. 1, pp. 135–146, 2007.
5. WEKA Tool Documentation, University of Waikato, New Zealand. Available: https://www.cs.waikato.ac.nz/ml/weka/
6. Kaggle Dataset Repository. Available: https://www.kaggle.com/
7. Pedregosa, F., et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
8. Tan, P.-N., Steinbach, M., & Kumar, V., *Introduction to Data Mining*, Pearson Education, 2006.